

HOSPITAL MANAGEMENT SYSTEM

System Architecture and Design Document

Version 1.0 | April 2026

Project Title	Hospital Management System
Document Type	Architecture & Design Document
Version	1.0
Authors	Ukkasha Ahmed, Yesaullah Sheikh
Date	April 2026

1. Introduction

1.1 Purpose

This System Architecture and Design Document (SADD) describes the structural and detailed design of the Hospital Management System (HMS). It covers the architectural style, system decomposition into layers and components, database design, module interactions, security design, and deployment considerations. This document bridges the Software Requirements Specification (SRS) and the implementation phase.

1.2 Scope

This document covers the complete design of the HMS, a Flask-based web application that manages hospital operations across five user roles. It describes the architectural decisions made during design and provides sufficient detail for developers to implement, test, and maintain the system.

1.3 Design Goals

- Separation of concerns through a layered architectural pattern
- Maintainability through the Flask Blueprint modular structure
- Security through role-based access control enforced at the route layer
- Data integrity through SQLAlchemy ORM with transactional operations
- Extensibility: database backend switchable via a single environment variable

2. Architectural Overview

2.1 Architectural Style

The HMS follows a three-tier, Model-View-Controller (MVC) architectural pattern delivered as a monolithic server-rendered web application. The three tiers are:

Tier	Layer	Technology	Responsibility
Presentation	View	Jinja2 HTML Templates	Renders UI; receives user input; displays data returned by controllers
Application	Controller	Flask Blueprints + Routes	Handles HTTP requests; enforces authentication/RBAC; orchestrates business logic
Application	Model / Business Logic	SQLAlchemy ORM Models	Defines data structures; contains business methods (dispense, discharge, calculate_total)
Data	Persistence	SQLite / MySQL / PostgreSQL	Stores and retrieves all persistent data via parameterised ORM queries

2.2 High-Level System Architecture

The following describes the system layers from client to data store:

Layer 1 — Client (Browser)
Any modern browser (Chrome, Firefox, Edge) running on hospital staff workstations.
Sends HTTP GET and POST requests. Receives and renders HTML responses.
No client-side JavaScript framework; all rendering is server-side via Jinja2.

Layer 2 — Web Server / WSGI
Development: Flask built-in development server (app.run(debug=True)).
Production: Gunicorn WSGI server (gunicorn wsgi:app).
Handles TCP connections, request parsing, and passes WSGI requests to Flask.

Layer 3 — Application Layer (Flask)
Flask Application Factory: create_app() initialises extensions and registers blueprints.
Flask-Login: Session management, @login_required enforcement.
Flask-WTF: CSRF-protected form handling and server-side validation.

Flask-Bcrypt: Password hashing at registration; hash comparison at login.
Flask-JWT-Extended: JWT issuance for API access (future integration).
Flask-CORS: Cross-origin request headers for browser compatibility.
Seven Blueprints: auth, main, patients, doctors, nurses, receptionists, store.

Layer 4 — ORM / Data Access Layer
Flask-SQLAlchemy (SQLAlchemy 2.0): Maps Python model classes to relational tables.
All queries use ORM session methods (<code>query.filter_by</code> , <code>session.add</code> , <code>session.commit</code>).
Flask-Migrate (Alembic): Schema version control; migrations in <code>/migrations/versions/</code> .
No raw SQL strings; parameterised queries enforced by ORM to prevent SQL injection.

Layer 5 — Data Store
Default: SQLite (file: <code>instance/hospital.db</code>). Zero-configuration; suitable for development.
Production: MySQL or PostgreSQL, configured via <code>DATABASE_URL</code> environment variable.
Schema: 20 tables covering users, patients, appointments, admissions, rooms, bills, prescriptions, inventory, orders, and supporting entities.

3. Module Design

3.1 Application Factory (app/__init__.py)

The create_app() factory function is the system entry point. It is responsible for:

- Loading environment variables from .env via python-dotenv
- Configuring Flask settings (SECRET_KEY, DATABASE_URL, JWT_SECRET_KEY)
- Initialising all Flask extensions: SQLAlchemy, Bcrypt, LoginManager, JWTManager, CORS, Migrate
- Registering all seven Blueprints
- Creating all database tables via db.create_all() within the application context

3.2 Blueprint Modules

Blueprint	Module Path	Key Responsibilities	Forms Used
auth	app/auth/	User registration (hashed password), login (session creation), logout, account management	RegistrationForm, LoginForm
main	app/main/	Public home page and about page routing	None
patients	app/patients/	Patient registration, profile view, medical record management, appointment scheduling from patient side	PatientRegistrationForm, AppointmentForm
doctors	app/doctors/	Doctor profile management, appointment queue view, patient detail view, prescription creation, appointment status updates	DoctorProfileForm, PrescriptionForm
nurses	app/nurses/	Nurse profile management, patient assignment management, vital signs recording (PatientStatus), room overview	NurseProfileForm, AssignmentForm, PatientStatusForm
receptionists	app/receptionists/	Patient registration, appointment scheduling/editing, patient admission, room management, discharge, bill generation	ReceptionistProfileForm, BillForm, AppointmentEditForm, DischargeForm
store	app/store/	Inventory CRUD, supplier management, purchase order creation and status management	InventoryForm, SupplierForm, OrderForm

3.3 Model Layer Design (app/models.py)

All database models inherit from db.Model (SQLAlchemy). Key design patterns in the model layer:

3.3.1 User and Role Profile Pattern

The User model stores credentials and role only. Each role has a separate profile model (Doctor, Nurse, Receptionist, StoreManager) linked via a user_id foreign key. This separation keeps the User model lightweight and allows role profiles to have domain-specific fields without polluting a single large user table.

Model	Key Fields	Business Methods
User	id, username, email, password_hash, role	check_password(password): Compares bcrypt hash
Doctor	user_id, specialization, qualification, experience_years, office_room	None (data model)
Nurse	user_id, department	None (data model)
Receptionist	user_id, first_name, last_name, contact	None (data model)
StoreManager	user_id, first_name, last_name, contact	None (data model)

3.3.2 Patient Workflow Models

Model	Key Fields	Business Methods / Notes
Patient	first_name, last_name, dob, gender, blood_group, contact_number, address	Central entity; referenced by Appointments, MedicalRecords, Prescriptions, Admissions
Appointment	patient_id, doctor_id, date, status, is_inpatient, room_id, nurse_id	assign_room(room_id, nurse_id): Sets room occupied; discharge(): Frees the room
Admission	patient_id, room_id, doctor_id, nurse_id, admission_date, discharge_date, reason, status	Tracks full inpatient lifecycle; status: admitted / discharged
Room	room_number, room_type, status, daily_rate	set_available(), set_occupied(), set_maintenance(): Enforce valid state transitions
Bill	admission_id / appointment_id, room_charges, medical_charges, medication_charges, other_charges, total, payment_status	calculate_total(): Sums all charge fields and assigns to total_amount
MedicalRecord	patient_id, diagnosis, treatment, notes, record_date	Append-only clinical record; created by doctors

Model	Key Fields	Business Methods / Notes
PatientStatus	patient_id, nurse_id, status, blood_pressure, temperature, pulse, timestamp	Immutable vitals log; one record per observation

3.3.3 Prescription and Medication Models

Model	Key Fields	Business Methods
Prescription	patient_id, doctor_id, prescription_date, notes	Parent record grouping medications prescribed in one visit
PrescriptionMedication	prescription_id, medication_id, dosage, frequency, duration, quantity, dispensed, dispensed_at	dispense(): Checks stock >= quantity; deducts stock; marks dispensed=True
Medication	name, description, manufacturer, unit_price, stock_quantity	Stock is decremented by PrescriptionMedication.dispense()

3.3.4 Inventory and Supply Chain Models

Model	Key Fields	Notes
Inventory	item_name, category, unit_price, stock_quantity, reorder_level, supplier_id	Categories: medication, equipment, supplies; reorder_level triggers manual reorder alert
MedicalProcedure	patient_id, doctor_id, procedure_name, procedure_date	Links to ProcedureInventoryItem; tracks which inventory was consumed
ProcedureInventoryItem	procedure_id, inventory_id, quantity_used	use_items(): Deducts quantity_used from Inventory.stock_quantity
Supplier	name, contact_person, contact_number, email, address	Referenced by Inventory items and Orders
Order	supplier_id, order_date, status, total_amount, notes	Status lifecycle: pending -> delivered / cancelled
OrderItem	order_id, inventory_id, quantity, unit_price	Individual line items in a purchase order

4. Database Design

4.1 Database Technology

The HMS uses SQLite as the development database (stored at instance/hospital.db). In production, the system can be configured to use MySQL or PostgreSQL by setting the DATABASE_URL environment variable. All schema changes are managed through Flask-Migrate (Alembic), ensuring schema version control across environments.

4.2 Entity-Relationship Summary

The database consists of 20 tables with the following key foreign-key relationships:

Relationship	Type	Description
User -> Doctor / Nurse / Receptionist / StoreManager	One-to-One	Each User has at most one role-specific profile
Patient -> Appointment	One-to-Many	A patient can have multiple appointments
Doctor -> Appointment	One-to-Many	A doctor can have multiple appointments
Appointment -> Room	Many-to-One	An inpatient appointment is assigned one room
Patient -> Admission	One-to-Many	A patient can have multiple admissions over time
Admission -> Room	Many-to-One	An admission occupies one room
Admission -> Bill	One-to-Many	One admission can have multiple bills (itemised)
Appointment -> Bill	One-to-Many	One outpatient appointment can have a bill
Doctor -> Prescription	One-to-Many	A doctor issues multiple prescriptions
Prescription -> PrescriptionMedication	One-to-Many	A prescription contains multiple medication items
PrescriptionMedication -> Medication	Many-to-One	Each prescription line references one medication
Nurse -> NurseAssignment -> Patient	Many-to-Many (via junction)	Nurses assigned to multiple patients
Inventory -> Supplier	Many-to-One	Each inventory item has a supplier
Order -> OrderItem -> Inventory	One-to-Many / Many-to-One	Purchase orders contain line items referencing inventory

4.3 Key Database Constraints

- Primary keys are auto-incremented integers on all tables
- NOT NULL constraints are applied to all business-critical fields (patient name, appointment date, room number)
- UNIQUE constraint on User.username and User.email
- UNIQUE constraint on Room.room_number
- Foreign keys enforce referential integrity; cascade='all, delete-orphan' on all relationship definitions ensures child records are deleted with parent
- Default values: Appointment.status = 'scheduled'; Room.status = 'available'; Order.status = 'pending'; Bill.payment_status = 'pending'

5. Security Design

5.1 Authentication Flow

The authentication flow proceeds as follows:

Step 1 — Registration
User submits username, email, password, and role via RegistrationForm.
Flask-Bcrypt hashes the password: <code>bcrypt.generate_password_hash(password).decode('utf-8')</code> .
User record is saved with <code>password_hash</code> ; plain-text password is discarded immediately.

Step 2 — Login
User submits email and password via LoginForm.
System retrieves User by email; calls <code>user.check_password(password)</code> which uses <code>bcrypt.check_password_hash</code> .
On success: <code>login_user(user)</code> creates a signed session cookie via Flask-Login.
On failure: generic error message shown; no information about which field is wrong (prevents user enumeration).

Step 3 — Session Management
All subsequent requests carry the session cookie.
Flask-Login's <code>@login_required</code> decorator validates the session before executing route handlers.
<code>current_user</code> proxy provides the authenticated User object to all route handlers.

5.2 Role-Based Access Control (RBAC)

Every role-specific route handler begins with an explicit role check:

RBAC Guard Pattern (applied to every protected route)
<code>if current_user.role != 'doctor': # Replace with the required role</code>
<code> flash('You do not have permission to view this page', 'danger')</code>
<code> return redirect(url_for('main.home'))</code>
This guard is applied manually at the start of every route handler in each blueprint.
A user who manipulates their URL cannot access another role's views; the check is server-side.

Role	Access Scope	Cannot Access
Doctor	Own appointments, own patients (via appointments), prescriptions	Other doctors' patients, billing, inventory, admin functions
Nurse	Assigned patients, patient status updates, room overview	Appointments, billing, prescriptions, inventory
Receptionist	All patients, appointments, admissions, rooms, billing	Prescriptions, inventory orders, vital sign entry
Store Manager	Inventory, suppliers, purchase orders	Patient data, appointments, billing, prescriptions
Patient	Own appointments, own medical records	Other patients, billing, doctor/nurse functions

5.3 Input Validation and CSRF Protection

- All forms inherit from FlaskForm (Flask-WTF), which automatically includes a CSRF token in the rendered HTML and validates it on submission
- WTForms validators (DataRequired, Email, Length, NumberRange) enforce field-level validation server-side before any database operation
- SQLAlchemy ORM parameterised queries prevent SQL injection at the data access layer
- File uploads are not part of the current scope, eliminating file-based attack vectors

6. Key Workflow Designs

6.1 Patient Admission Workflow

Step	Actor	Component	Action / System Response
1	Receptionist	Browser -> receptionists blueprint	Navigates to /receptionists/patients/<id>/admit
2	System	receptionists/routes.py	Queries available rooms, active doctors, and nurses; renders admission form
3	Receptionist	Browser (form submission)	Submits: room_id, doctor_id, nurse_id, reason, notes
4	System	receptionists/routes.py	Creates Admission record; sets room.status = 'occupied' (Room.set_occupied())
5	System	SQLAlchemy / SQLite	db.session.commit() — transaction commits atomically; rollback on exception
6	System	Browser (Jinja2 redirect)	Flash success message; redirect to patient profile showing 'Admitted' status

6.2 Prescription and Dispensing Workflow

Step	Actor	Component	Action / System Response
1	Doctor	Browser -> doctors blueprint	Navigates to /doctors/prescriptions/new/<patient_id>
2	System	doctors/routes.py	Queries available medications; renders PrescriptionForm
3	Doctor	Browser (form)	Enters notes; adds medication items (dosage, frequency, duration, quantity)
4	System	doctors/routes.py	Creates Prescription record; creates PrescriptionMedication records (dispensed=False)
5	Store/Nurse	Browser -> store/nurses blueprint	Navigates to dispense action for the prescription item
6	System	PrescriptionMedication.dispatch()	Checks medication.stock_quantity >= quantity; deducts stock; sets dispensed=True; records dispensed_at timestamp

6.3 Billing Workflow

Step	Actor	Component	Action / System Response
1	Receptionist	Browser -> receptionists blueprint	Navigates to /receptionists/admissions/<id>/bill or /receptionists/appointments/<id>/bill
2	System	receptionists/routes.py	Renders BillForm pre-populated with admission/appointment reference
3	Receptionist	Browser (form)	Enters room_charges, medical_charges, medication_charges, other_charges
4	System	Bill.calculate_total()	Sums all charge fields; assigns to total_amount
5	System	SQLAlchemy	Persists Bill record; sets payment_status = 'pending'
6	System	Browser (Jinja2)	Displays final bill with total amount and payment status

7. Deployment Design

7.1 Development Environment

Component	Configuration
Entry Point	app.py — calls create_app() and runs Flask dev server on port 5000
Database	SQLite (instance/hospital.db) — auto-created by db.create_all() on first run
Configuration	.env file with SECRET_KEY, DATABASE_URL (optional), JWT_SECRET_KEY
Schema Init	flask db upgrade — applies Alembic migrations; seed_rooms.py seeds room data
Dependencies	pip install -r requirements.txt

7.2 Production Environment

Component	Configuration
WSGI Server	Gunicorn: gunicorn --workers 4 --bind 0.0.0.0:8000 'app:create_app()'
Database	MySQL or PostgreSQL via DATABASE_URL environment variable
Reverse Proxy	Nginx (recommended) to terminate HTTPS and proxy to Gunicorn
Environment Variables	SECRET_KEY (high-entropy random string), JWT_SECRET_KEY, DATABASE_URL
Session Security	SESSION_COOKIE_SECURE=True, SESSION_COOKIE_HTTPONLY=True in production config

7.3 Directory Structure

Project Directory Structure	
HMS/	
— app.py	# Application entry point
— requirements.txt	# Python dependencies
— .env	# Environment variables (not committed to VCS)
— seed_rooms.py	# Room data seeding script
— instance/	
— hospital.db	# SQLite database file
— migrations/	# Alembic migration files
— versions/	# Individual migration scripts

└─ app/	
├─ __init__.py	# Application factory (create_app)
├─ models.py	# All SQLAlchemy ORM models
├─ auth/	# Authentication blueprint
├─ main/	# Public pages blueprint
├─ patients/	# Patient management blueprint
├─ doctors/	# Doctor functions blueprint
├─ nurses/	# Nurse functions blueprint
├─ receptionists/	# Receptionist functions blueprint
├─ store/	# Store manager blueprint
└─ templates/	# Jinja2 HTML templates (per blueprint)

8. Design Patterns and Principles

Pattern / Principle	Application in HMS
Application Factory Pattern	<code>create_app()</code> function creates the Flask app, initialises extensions, and registers blueprints. Enables testing with different configurations without reinstantiating the global app object.
Blueprint Pattern (Modular MVC)	Each role has its own Blueprint with isolated routes, forms, and templates. Reduces coupling between modules; each blueprint can be developed and tested independently.
Repository Pattern (via ORM)	All database interactions go through SQLAlchemy session methods. Route handlers never write raw SQL; the ORM acts as the data access layer.
State Machine Pattern	Room model implements explicit state transition methods (<code>set_available</code> , <code>set_occupied</code> , <code>set_maintenance</code>) with guard conditions, preventing invalid state jumps.
Active Record Pattern	Business methods (<code>dispense</code> , <code>discharge</code> , <code>calculate_total</code> , <code>use_items</code>) live on the model classes rather than in service layers, following SQLAlchemy's active-record convention for small applications.
Template Method Pattern	Jinja2 templates extend <code>base.html</code> using template inheritance, ensuring consistent navigation, flash messages, and layout without code duplication.
RBAC Guard Pattern	Role verification at the top of every route handler enforces least-privilege access. Centralising this check in the route layer makes it auditable.
Separation of Concerns	Forms (validation), Routes (controller logic), Models (data/business logic), and Templates (presentation) are kept in separate modules, enabling independent modification of each layer.

9. Non-Functional Design Considerations

9.1 Performance

- SQLAlchemy lazy loading is the default on all relationships, preventing N+1 queries on list views
- Pagination should be applied to patient and appointment list views for large datasets using Flask-SQLAlchemy's `paginate()` method
- Database indexes on frequently queried foreign key columns (`patient_id`, `doctor_id`, `appointment_date`) improve query performance
- Gunicorn with multiple workers enables concurrent request handling in production

9.2 Maintainability

- Flask-Migrate (Alembic) ensures all schema changes are versioned and reproducible; no manual `ALTER TABLE` statements
- Blueprint isolation means a change to the receptionist module does not affect the doctor or store modules
- The `models.py` single-file design keeps all entity definitions together for easy reference and cross-model relationship inspection
- Environment variable-based configuration (`python-dotenv`) separates configuration from code

9.3 Scalability

- Switching from SQLite to MySQL/PostgreSQL by updating `DATABASE_URL` is the primary scaling step; no code changes required
- The application factory pattern supports deployment of multiple Gunicorn workers or processes
- Stateless JWT tokens (in addition to Flask-Login sessions) enable future API clients without server-side session storage

9.4 Testability

- The application factory pattern allows test configurations to use an in-memory SQLite database, isolating tests from production data
- Each blueprint's routes are independently importable, enabling unit testing of individual route handlers
- WTForms validation logic is testable independently of the HTTP layer by instantiating forms with test data